

RAPPORT DES PROJETS FINAL

Weather ETL Platform & Analyse des données météorologiques

Pipeline Extract · Transform · Load, puis exploration, statistiques et apprentissage automatique

Étude appliquée à l'historique météorologique de Szeged (Hongrie), 2006–2016

Data Engineering SIGUI DE6 ForceN, Groupe A

Présenté par :

KONE Abdel Kader Razack

COULIBALY Laure Elodie

FATOU NIANH

Mouhamed Cissé

Mourtalla Toure

Nafi GUEYE

GBEUNEYALLA Valentin

Mbayame GUEYE

Thierno BALDE

BONHEUR BEHIDI DONANG

Aïssatou BA

Ousseynou DIOUF

Baptiste Roby

Table des matières

Table des matières	2
Introduction générale	3
Partie I : Ingénierie des données (ETL)	5
1. Contexte et objectifs	5
2. Architecture globale	5
3. Stack technique (Docker)	7
4. Extract : récupération du dataset	7
5. Ingestion vers MinIO.....	7
6. NiFi : configuration des processeurs	7
7. Transform : nettoyage et normalisation	8
8. Stockage MinIO	9
9. Load : Data Warehouse MongoDB	10
10. Automatisation.....	11
11. Exploitation des résultats.....	11
12. Difficultés rencontrées (équipe ETL)	11
Partie II : Analyse des données.....	13
1. Origine et préparation des données	13
2. Analyse exploratoire et statistique	13
3. Représentation graphique et tableau de bord	14
4. Apprentissage automatique	15
Livrables	18
Conclusion générale.....	18
Références	20

Introduction générale

L'ingénierie des données (Data Engineering) occupe une place centrale dans la valorisation moderne de l'information : avant qu'une donnée puisse être analysée, visualisée ou exploitée par un modèle prédictif, elle doit être collectée, nettoyée, structurée et rendue durablement accessible. C'est autour de cette chaîne complète de la donnée brute à la donnée exploitable que s'articule le présent projet, réalisé dans le cadre de la formation Data Engineering du programme ForceN, porté par l'Université Numérique Cheikh Hamidou Kane.

Le projet s'appuie sur un cas d'usage concret et pédagogiquement riche : l'historique météorologique horaire de la ville de Szeged, en Hongrie, couvrant la période 2006–2016 et comptant 96 453 relevés répartis sur 12 variables (température, humidité, vent, pression, précipitations, etc.). Ce jeu de données présente l'ensemble des difficultés typiques rencontrées en environnement réel : valeurs manquantes, erreurs de capteur, colonnes redondantes ou non informatives, et une forte composante temporelle qu'il convient de traiter avec rigueur. Il constitue à ce titre un terrain d'application représentatif des problématiques auxquelles un ingénieur de données est couramment confronté.

Pour mener ce projet à bien, le Groupe A s'est organisé en deux équipes complémentaires, opérant sur les deux moitiés d'une même chaîne de traitement. Une équipe ETL (Extract, Transform, Load) a pris en charge l'extraction des données depuis Kaggle, leur nettoyage et leur normalisation, ainsi que leur stockage structuré dans une infrastructure combinant un data lake (MinIO) et un entrepôt de données documentaire (MongoDB), le tout orchestré et automatisé à l'aide de Docker, Apache NiFi et de scripts Python. Une équipe Analyse a ensuite pris le relais sur les données ainsi préparées, afin d'en explorer les tendances, d'en vérifier statistiquement les relations, de les restituer visuellement au travers d'un tableau de bord interactif, et enfin de construire un modèle d'apprentissage automatique capable de prédire la température.

Le présent document a pour vocation de rassembler, dans un rapport unique et cohérent, l'ensemble des travaux réalisés par ces deux équipes. Plutôt que de juxtaposer deux livrables distincts, il propose une lecture continue de la chaîne de valeur de la donnée : de l'extraction brute jusqu'à la prédiction, en passant par le nettoyage, le stockage et l'analyse. Cette approche permet de mettre en évidence la dépendance directe entre la qualité du travail d'ingénierie effectué en amont et la fiabilité des résultats analytiques et prédictifs obtenus en aval.

Le rapport est structuré en deux grandes parties, suivies d'une synthèse transversale. La Partie I détaille l'ensemble de la chaîne ETL : architecture technique, extraction, ingestion, transformation, stockage et chargement des données, ainsi que les difficultés rencontrées et les livrables produits. La Partie II présente le travail de l'équipe Analyse : préparation finale du jeu de données, analyse exploratoire et statistique, visualisation et tableau de bord, puis modélisation prédictive. Le document se conclut par une synthèse globale du pipeline, la liste consolidée des livrables, et une conclusion générale mettant en perspective l'ensemble des résultats obtenus. Cette version est soumise pour relecture et correction avant sa finalisation, et sera complétée par une présentation de synthèse.

Partie I : Ingénierie des données (ETL)

1. Contexte et objectifs

L'objectif de la contribution ETL est de transformer le dataset météo brut, tel que publié sur Kaggle, en données propres, structurées et exploitables pour l'équipe Analyse. Au-delà du simple nettoyage, l'ambition affichée par l'équipe est de livrer un pipeline reproductible, documenté et prêt à la passation : toute personne disposant du dépôt de code doit pouvoir reconstituer l'intégralité de la chaîne de traitement, du téléchargement des données brutes jusqu'à leur disponibilité dans l'entrepôt MongoDB, sans intervention manuelle. Le tableau suivant détaille la correspondance entre chaque contribution de l'équipe et l'objectif qu'elle poursuit.

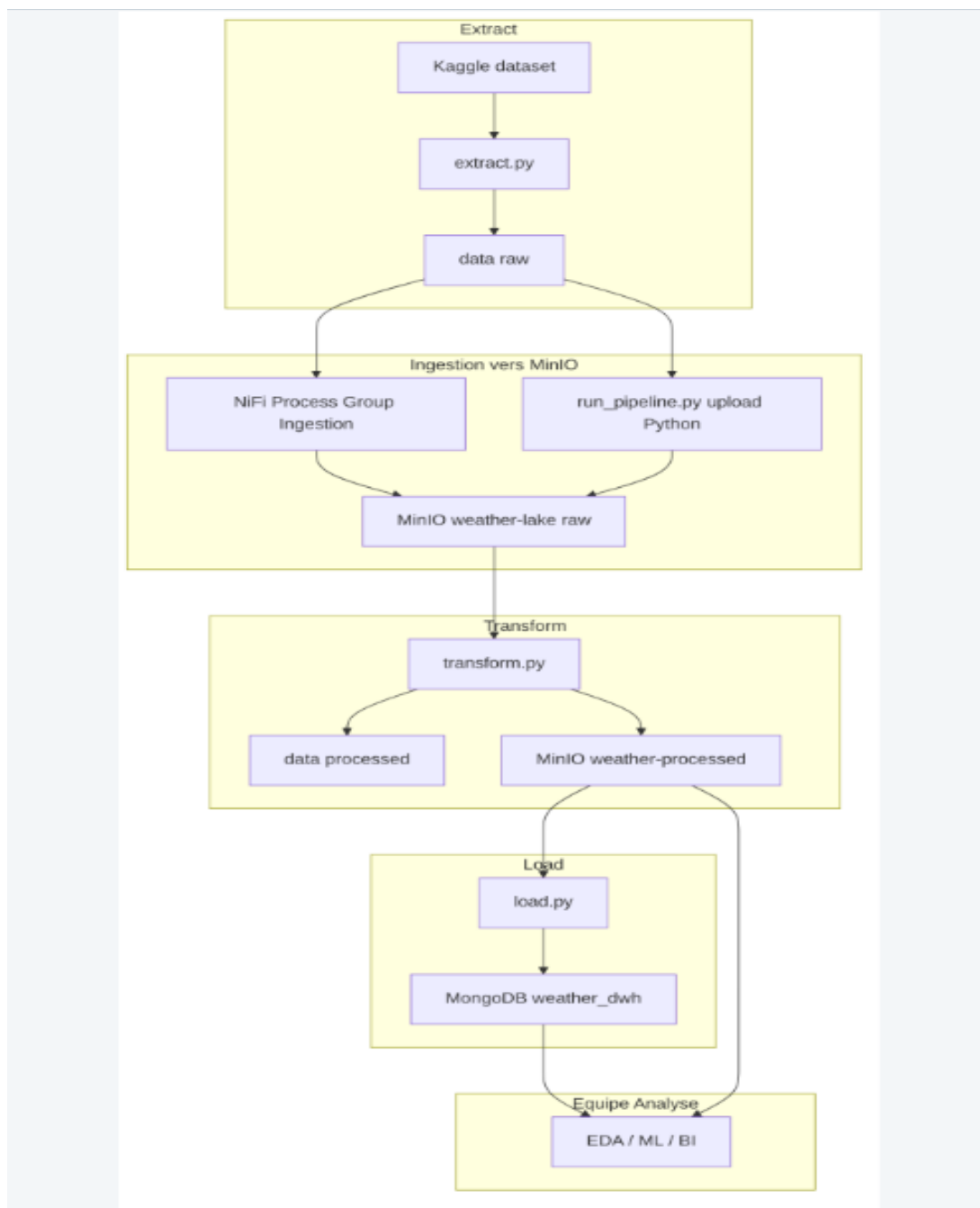
Contribution ETL	Objectif
Extraction du dataset météo depuis Kaggle	Récupérer les données sources brutes
Stockage raw dans MinIO (weather-lake)	Conserver une zone de données originales
Transformation avec Python / pandas	Nettoyer, normaliser et préparer les données
Stockage des données transformées dans MinIO (weather-processed)	Conserver une zone de données nettoyées prêtes à l'exploitation
Chargement dans MongoDB (weather_dwh)	Mettre à disposition un Data Warehouse exploitable
Automatisation et documentation	Rendre le pipeline reproductible et faciliter sa réutilisation

2. Architecture globale

Le pipeline repose sur une architecture en quatre grandes étapes, organisées de façon strictement séquentielle : Extract → Ingestion MinIO → Transform → Load MongoDB → Analyse / Visualisation / Modélisation. Le jeu de données est d'abord extrait depuis Kaggle par un script Python (extract.py), puis déposé sous sa forme brute dans une zone dédiée du data lake MinIO (le bucket weather-lake), qui constitue une copie de référence immuable des données telles que collectées. Cette étape d'ingestion peut être réalisée de deux façons complémentaires : soit via un flux Apache NiFi, qui illustre une orchestration visuelle et industrialisable de l'ingestion, soit via un script Python autonome, utilisé pour l'exécution automatisée du pipeline complet.

Les données brutes sont ensuite lues depuis MinIO par le script de transformation (transform.py), qui applique l'ensemble des règles de nettoyage, de normalisation et d'enrichissement détaillées à

la section 8, avant d'écrire le résultat dans une seconde zone du data lake (le bucket weather-processed), séparée de la zone brute afin de préserver une distinction claire entre données sources et données dérivées. Enfin, le script de chargement (load.py) lit les données transformées et les charge dans une base MongoDB (weather_dwh), qui constitue l'entrepôt de données exploitable par l'équipe Analyse pour ses travaux d'exploration, de visualisation et de modélisation. Cette séparation en zones successives (raw, processed, entrepôt) garantit la traçabilité de chaque transformation et permet, en cas d'anomalie détectée en aval, de revenir à tout moment aux données sources sans avoir à relancer l'extraction depuis Kaggle.



3. Stack technique (Docker)

Trois services sont orchestrés par docker-compose.yml, démarrables en une seule commande (python scripts/setup.py) :

Service	Image	Port(s)	Rôle
MongoDB	mongo:latest	27017	Data Warehouse
MinIO	minio/minio	9000 / 9001	Data Lake (S3)
NiFi	apache/nifi	8443	Orchestration ingestion

4. Extract : récupération du dataset

Le script etl/extract.py télécharge le dataset via kagglehub (source : muthuj7/weather-dataset) et le copie dans data/raw. Le fichier weatherHistory.csv obtenu comporte 96 453 lignes brutes réparties sur 12 colonnes.

5. Ingestion vers MinIO

Deux mécanismes écrivent le CSV brut dans le bucket weather-lake, chacun répondant à un usage distinct :

Mécanisme	Usage	Destination
NiFi GetFile → PutS3Object	Démonstration d'orchestration	weather-lake/raw/
Python (upload_raw_to_minio)	Pipeline automatisé (scripts/run_pipeline.py)	weather-lake/raw/

Le flux NiFi correspondant enchaîne les étapes suivantes : le fichier produit par extract.py est déposé dans un volume d'entrée surveillé par NiFi, un premier processeur GetFile en récupère le contenu, puis un second processeur PutS3Object le dépose dans le bucket weather-lake de MinIO via une connexion compatible S3. Ce flux est décrit plus en détail à la section suivante.

6. NiFi : configuration des processeurs

Le flux NiFi récupère les fichiers bruts depuis data/raw et les dépose automatiquement dans MinIO via une connexion compatible S3 :

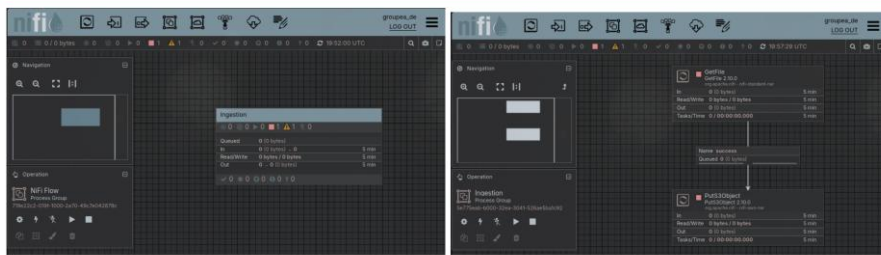
Processeur	Configuration clé
GetFile	Input Directory : /opt/nifi/input

PutS3Object	Bucket : weather-lake · Object Key : raw/\${filename} · Endpoint : http://minio:9000
-------------	---

Le Controller Service `AWSCredentialsProviderControllerService` doit être configuré avec `MINIO_ACCESS_KEY` et `MINIO_SECRET_KEY`. La figure suivante illustre l'état du Process Group NiFi correspondant, avant et après exécution du flux `GetFile` → `PutS3Object`.

Weather ETL Platform — ForceN Groupe A

NiFi — Process Group



Guide pas à pas complet : [rapport/nifi-ingestion.md](#)

Figure 1 : NiFi : Process Group d'ingestion (`GetFile` → `PutS3Object`)

Guide pas à pas complet : [rapport/nifi-ingestion.md](#).

7. Transform : nettoyage et normalisation

Le script `etl/transform.py` lit les données brutes depuis MinIO, applique le nettoyage puis écrit le résultat localement et dans le bucket `weather-processed`. Les traitements appliqués sont :

- Normalisation des noms de colonnes en `snake_case`
- Parsing des dates en UTC (`date_time`)
- Suppression des doublons et des lignes sans mesures essentielles
- Nettoyage du texte (`summary`, `precip_type`, `daily_summary`)
- Traitement des valeurs manquantes : interpolation temporelle et médiane
- Correction des valeurs numériques incohérentes (ex. pression à 0 mbar)
- Suppression de la colonne `loud_cover` (valeur constante, sans information)
- Enrichissement (version `ENRICHED`) : ajout de colonnes temporelles dérivées

Impact global du traitement sur le jeu de données :

Indicateur	Avant	Après
Lignes	96 453	96 429
Colonnes	12	11
Doublons	Présents	Supprimés
Valeurs manquantes	Nombreuses	Réduites / imputées
Valeurs numériques incohérentes	Présentes	Corrigées
Colonne constante	Présente	Retirée

Le dataset devient plus propre, plus stable et plus simple à charger dans MongoDB. Cinq fichiers sont produits dans le bucket weather-processed, chacun répondant à un usage spécifique en aval :

Fichier	Emplacement MinIO	Utilisation
weather_processed.csv	weather-processed/	Source du load MongoDB
weather_processed.parquet	weather-processed/	Format columnar
weather_processed_enriched.csv	weather-processed/	+ features temporelles
weather_processed_enriched.parquet	weather-processed/	Version Parquet enrichie
weatherHistory_clean_metadata.json	weather-processed/	Métadonnées du run

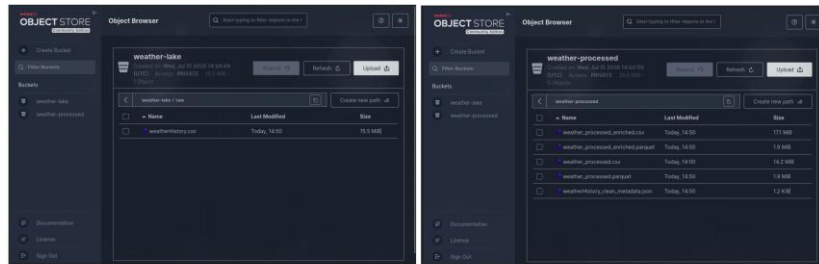
Copie locale : data/processed/weather_processed.csv.

8. Stockage MinIO

Deux buckets séparent la zone brute et la zone préparée : weather-lake (raw) et weather-processed (CSV, Parquet, métadonnées). Cette organisation en zones successives, visible dans l'interface d'administration de MinIO ci-dessous, matérialise concrètement le principe de séparation entre données sources et données dérivées évoqué à la section 2.

MinIO — Stockage

Deux buckets séparent zone raw et zone curated :



weather-lake (raw) · **weather-processed** (CSV, Parquet, metadata)

Figure 2 : Object Browser MinIO : buckets weather-lake et weather-processed

9. Load : Data Warehouse MongoDB

Le script etl/load.py (pymongo) charge les données transformées dans MongoDB :

Élément	Valeur
Base / Collection	weather_dwh / weather_observations
Documents chargés	96 429
Mode	Remplacement complet (pas de doublons)
Index	date_time (unique), precip_type
Batch size	5 000 (configurable via MONGO_BATCH_SIZE)

Chaque document MongoDB correspond à un relevé horaire, structuré comme suit :

```
{
  "date_time": "2006-01-01T00:00:00Z",
  "summary": "Mostly Cloudy",
  "precip_type": "rain",
  "temperature_c": 1.16,
  "apparent_temperature_c": -3.24,
  "humidity": 0.85,
  "wind_speed_km_per_h": 16.62,
  "wind_bearing_degrees": 139.0,
  "visibility_km": 9.90,
  "pressure_millibars": 1016.15,
  "daily_summary": "Mostly cloudy throughout the day."
}
```

Une restauration de la base est fournie via mongodump / mongorestore. Instructions complètes : [rapport/database/README.md](#).

10. Automatisation

L'ensemble du pipeline est automatisé par des scripts protégés par fichier lock et vérification TCP des ports avant exécution, ce qui évite les exécutions concurrentes accidentelles et les échecs silencieux liés à un service non encore disponible :

Script	Rôle
scripts/setup.py	.env, environnement virtuel, Docker, attente des services
scripts/run_pipeline.py	Pipeline en 4 étapes (extract → upload → transform → load)
scripts/reset.py	Remise à zéro (data/, conteneurs ; option --full pour .venv)
rapport/scripts/export_assets.sh	Export CSV + mongodump vers rapport/

11. Exploitation des résultats

Deux options permettent d'utiliser les résultats du projet :

Option	Description
Clone GitHub	Environnement complet et reproductible avec setup.py puis run_pipeline.py
Archive rapport/	Datasets générés, dump MongoDB, documentation et instructions d'utilisation

Les fichiers volumineux (datasets CSV et dump MongoDB) ne sont pas versionnés dans Git. Ils sont générés après l'exécution complète du pipeline puis exportés dans le dossier rapport/. Après l'exécution du pipeline :

```
python scripts/setup.py
python scripts/run_pipeline.py
bash rapport/scripts/export_assets.sh
```

12. Difficultés rencontrées (équipe ETL)

Comme dans tout projet d'ingénierie de données, la mise en œuvre du pipeline s'est heurtée à plusieurs difficultés concrètes, résumées ci-dessous avec la solution retenue par l'équipe pour chacune d'elles.

Problème	Impact	Solution retenue
Configuration du processeur PutS3Object	Nécessité d'un AWSCredentialsProviderControllerService correctement configuré pour accéder à MinIO	Création et configuration du Controller Service avant l'exécution du flux
Transformations dans NiFi	Difficultés d'environnement et de dépendances pour exécuter directement le nettoyage dans NiFi	Regroupement des traitements dans etl/transform.py ; NiFi conservé pour l'ingestion vers MinIO
Définition des règles de transformation	Besoin d'analyser le dataset avant d'appliquer les traitements	Analyse exploratoire (EDA), recherche documentaire et validation des règles de nettoyage

Partie II : Analyse des données

Le présent volet résume le travail réalisé par l'équipe Analyse sur les données météorologiques du dataset Kaggle portant sur la ville de Szeged, en Hongrie, entre 2005 et 2016. Il s'inscrit dans la continuité directe du travail effectué par l'équipe ETL présenté en Partie I : c'est en effet sur le fichier `weather_processed_enriched.csv`, produit à l'issue de la chaîne de transformation, que reposent l'ensemble des analyses, visualisations et modèles présentés ci-après.

1. Origine et préparation des données

Les données brutes ont été collectées depuis Kaggle par l'équipe de collecte, puis nettoyées et stockées par l'équipe de traitement à l'aide d'un pipeline automatisé reposant sur Python, Docker, MinIO et MongoDB. Ce pipeline a corrigé 517 valeurs manquantes concernant le type de précipitations, supprimé 24 doublons, et corrigé 1 288 valeurs de pression atmosphérique physiquement impossibles, initialement enregistrées à 0 millibar. Le fichier résultant, `weather_processed_enriched.csv`, comporte 96 429 relevés horaires répartis sur 17 colonnes. Avant toute analyse, la qualité de ce fichier a été vérifiée de manière indépendante par l'équipe Analyse, confirmant l'absence de valeurs manquantes et de doublons résiduels et validant ainsi la fiabilité du travail réalisé en amont.

2. Analyse exploratoire et statistique

L'exploration des données a permis de dresser un premier portrait du jeu de données à travers des statistiques descriptives et des visualisations de distribution portant sur la température, l'humidité, la vitesse du vent et la pression atmosphérique. L'observation de la température moyenne mensuelle sur l'ensemble de la période a confirmé la cohérence du dataset avec un climat continental typique, marqué par une forte saisonnalité, avec des températures moyennes s'échelonnant d'environ 1,5 °C en hiver à environ 22 °C en été.

Cette exploration a ensuite été complétée par des tests statistiques destinés à vérifier la validité des relations observées. Une corrélation de Pearson entre la température et l'humidité a révélé un lien négatif fort et statistiquement significatif, avec un coefficient de -0,63 et une probabilité inférieure à 0,1 %. Une analyse de variance a par ailleurs confirmé que les écarts de température moyenne entre les quatre saisons n'étaient pas dus au hasard. Un test complémentaire a également montré que la vitesse du vent différait significativement selon que les précipitations se présentaient sous forme de pluie ou de neige. Une détection des valeurs aberrantes résiduelles par la méthode de

l'écart interquartile a enfin été menée sur le vent et la pression, révélant des valeurs extrêmes mais physiquement plausibles, qui n'ont donc pas été supprimées.

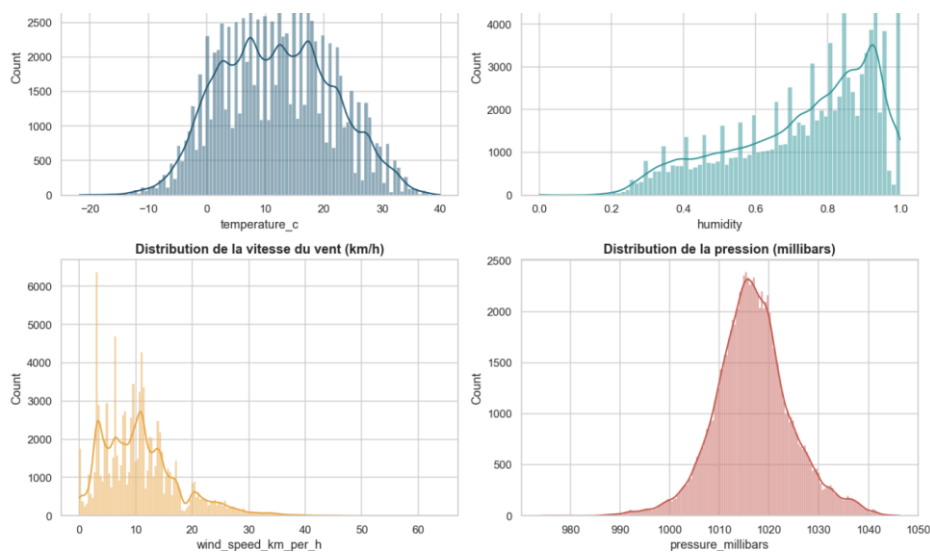


Figure 3 : Distribution des variables météorologiques

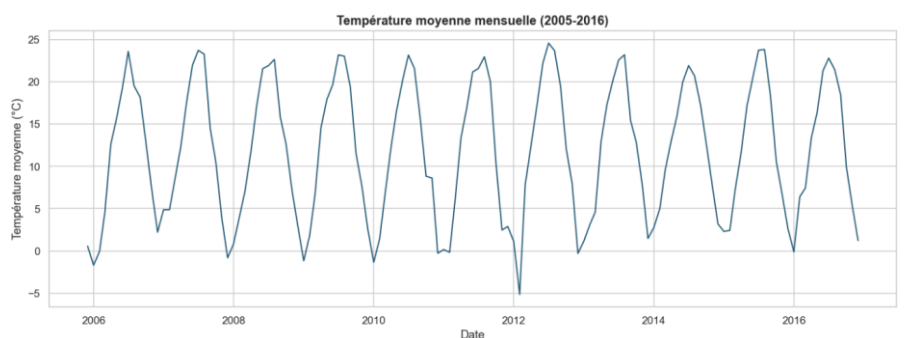


Figure 4 : Évolution de la température moyenne mensuelle (2005-2016)

3. Représentation graphique et tableau de bord

Une série de graphiques a été produite afin de représenter visuellement les distributions, les tendances et les relations entre variables du jeu de données. La distribution des températures enregistrées a été étudiée par histogramme, la relation entre température et humidité a été représentée selon la saison, et l'évolution des températures réelles et ressenties a été suivie sur l'ensemble de la période disponible.

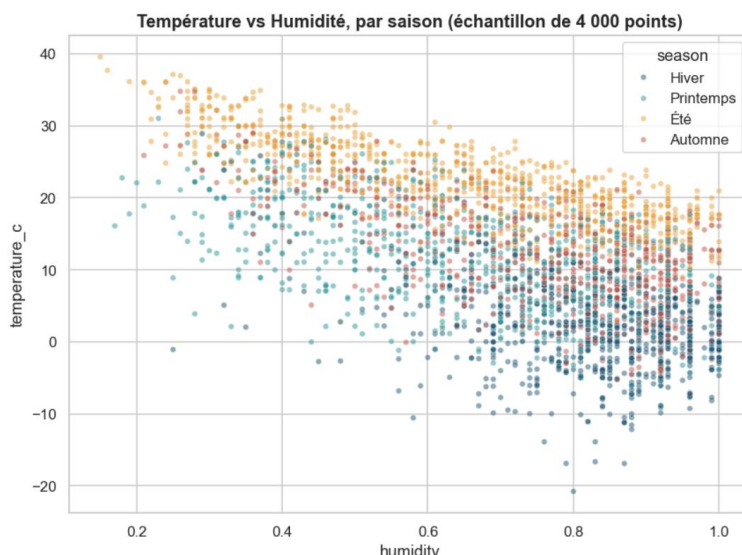


Figure 5 : Relation entre température et humidité selon la saison

Un tableau de bord interactif a par ailleurs été développé à l'aide de la bibliothèque Streamlit. Il permet de suivre en temps réel plusieurs indicateurs clés, dont la température moyenne, le taux d'humidité et le nombre total d'observations, avec la possibilité de filtrer les résultats par type de précipitations et par période. L'évolution journalière de la température ainsi que la distribution du taux d'humidité y sont représentées graphiquement et se mettent à jour automatiquement selon les filtres appliqués, ce qui permet à un utilisateur non technique d'explorer le jeu de données sans écrire de code.

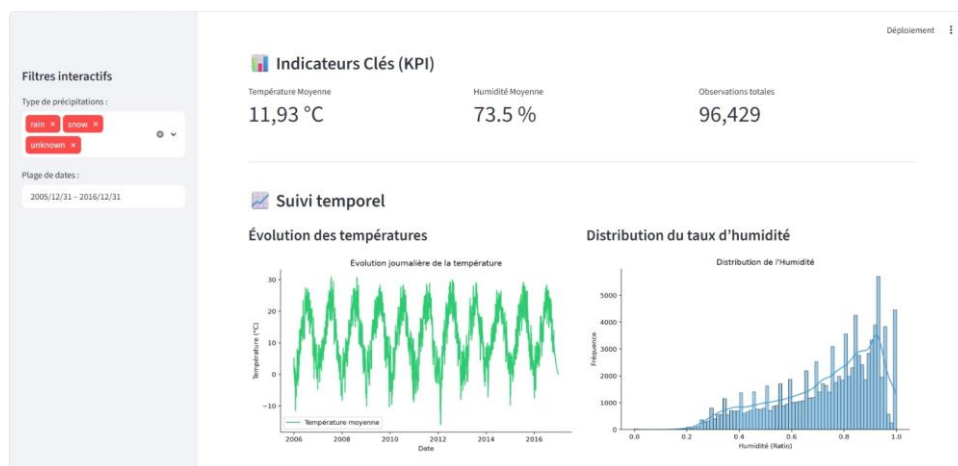


Figure 6 : Tableau de bord interactif Streamlit : indicateurs clés et suivi temporel

4. Apprentissage automatique

Au-delà de l'exploration descriptive et statistique, l'équipe Analyse a cherché à exploiter la qualité du jeu de données préparé par la chaîne ETL pour construire un modèle prédictif : un modèle de prédiction de la température a été développé à partir des variables disponibles dans le jeu de

données. Cette étape constitue le prolongement naturel du travail d'analyse : elle permet de vérifier, au-delà des corrélations descriptives mises en évidence précédemment (notamment le lien fort entre température et humidité), que ces relations sont suffisamment structurées et informatives pour être exploitées à des fins de prédiction, et non de simple description.

Le choix des variables explicatives n'a pas été laissé au hasard. Ont été retenues l'humidité, la vitesse du vent et la pression atmosphérique, trois grandeurs physiquement liées à la température et dont la pertinence avait déjà été établie lors de l'analyse statistique. À ces variables se sont ajoutées des variables temporelles construites spécifiquement pour l'occasion : l'heure et le mois, encodés de manière cyclique plutôt que sous forme de simples entiers. Ce choix méthodologique est important : un encodage naïf de l'heure (0 à 23) ou du mois (1 à 12) introduirait une discontinuité artificielle entre 23h et 0h, ou entre décembre et janvier, alors que ces instants sont en réalité très proches d'un point de vue climatique. Un encodage cyclique, typiquement au moyen de fonctions sinus et cosinus, restitue correctement cette continuité et évite au modèle d'apprendre une fausse rupture là où il n'y en a pas. Enfin, des variables retardées (lag features), correspondant aux températures des heures précédentes, ont été introduites afin de capter l'inertie thermique à court terme : la température d'une heure donnée est en effet fortement corrélée à celle des heures qui la précèdent immédiatement.

Sur le plan méthodologique, les données ont été réparties non pas aléatoirement, mais de façon strictement chronologique, en trois ensembles consacrés respectivement à l'entraînement (70 %), à la validation (15 %) et au test (15 %). Ce choix n'est pas anodin pour une série temporelle : un découpage aléatoire aurait pu placer, dans l'ensemble d'entraînement, des observations postérieures à celles de l'ensemble de test, ce qui aurait permis au modèle d'apprendre indirectement des informations futures via les variables retardées — un phénomène connu sous le nom de fuite de données (data leakage), qui aurait artificiellement gonflé les performances mesurées. Le découpage chronologique retenu ici reproduit au contraire une situation réaliste de prévision, dans laquelle le modèle n'a accès qu'au passé pour estimer le présent ou le futur proche.

Le modèle retenu est une forêt aléatoire (Random Forest), un algorithme d'ensemble reposant sur l'agrégation d'un grand nombre d'arbres de décision. Ce choix se justifie par plusieurs propriétés adaptées au contexte du projet : la forêt aléatoire ne fait aucune hypothèse de linéarité entre les variables explicatives et la variable cible, ce qui lui permet de capter des relations complexes ; elle est naturellement robuste au surapprentissage grâce au principe d'agrégation (bagging) sur lequel elle repose ; elle ne nécessite pas de normalisation préalable des variables, ce qui simplifie le pipeline de préparation ; et elle constitue un choix de référence solide, rapide à mettre en œuvre, avant

d'envisager des approches plus sophistiquées telles que le gradient boosting ou les réseaux de neurones.

Les résultats obtenus sont les suivants :

Indicateur	Valeur
Erreur moyenne absolue (MAE)	0,56 °C
Coefficient de détermination (R^2)	0,993
Répartition entraînement / validation / test	70 % / 15 % / 15 % (chronologique)

Une erreur moyenne absolue de 0,56 °C est, en pratique, remarquablement faible : elle est inférieure à la précision de lecture d'un grand nombre de capteurs météorologiques grand public, et donc parfaitement adaptée à un usage opérationnel. Le coefficient de détermination de 0,993 signifie que le modèle explique 99,3 % de la variance de la température observée sur l'ensemble de test. Il convient toutefois d'interpréter ce résultat avec la rigueur méthodologique qu'impose tout exercice de modélisation : un tel niveau de performance, bien que réel, s'explique en grande partie par la présence des variables retardées parmi les prédicteurs. La température atmosphérique présente en effet une forte autocorrélation à court terme — elle varie peu d'une heure à l'autre —, si bien qu'une part importante du pouvoir prédictif du modèle provient de cette persistance à court terme plutôt que d'une capacité à anticiper des ruptures climatiques. Ce résultat doit donc être lu comme la validation d'une excellente capacité du modèle à prolonger la dynamique récente des données, plus que comme une preuve de prévision à long horizon.

Cette nuance ouvre naturellement sur les limites et les perspectives d'amélioration du travail réalisé. Le modèle n'a été entraîné et validé que sur les données historiques de Szeged ; sa capacité de généralisation à d'autres zones géographiques ou à d'autres régimes climatiques resterait à démontrer. Sa dépendance aux variables retardées limite également son usage à un horizon de prévision très court, puisqu'il nécessite de disposer des températures les plus récentes pour produire une estimation. Plusieurs pistes d'amélioration pourraient être explorées dans la suite du projet : comparer les performances de la forêt aléatoire à celles d'algorithmes de gradient boosting (XGBoost, LightGBM) ou à celles d'une simple prévision de persistance utilisée comme référence basse ; conduire une analyse formelle de l'importance des variables afin d'identifier précisément la contribution de chaque prédicteur ; étendre le modèle à une prévision multi-horizon (à 6h, 12h ou 24h) plutôt qu'à l'instant suivant ; et enfin, envisager le déploiement du modèle sous forme de service exposé, alimenté en continu par les données chargées dans MongoDB, afin d'en faire un véritable outil de prévision opérationnel plutôt qu'un exercice académique isolé.

Synthèse globale du pipeline

Le tableau ci-dessous récapitule la chaîne complète, de l'extraction des données brutes jusqu'à la modélisation prédictive :

Étape	Technologie	Résultat
Extract	Python + Kaggle (kagglehub)	96 453 lignes brutes, 12 colonnes
Ingestion	NiFi + MinIO (ou upload Python)	weather-lake/raw/
Transform	Python + pandas	96 429 lignes nettoyées, 11 colonnes
Load	pymongo	96 429 documents indexés (weather_dwh)
Analyse	pandas, scipy, scikit-learn	Tests statistiques + modèle prédictif $R^2 = 0,993$
Visualisation	Matplotlib, Seaborn, Streamlit	4 figures + tableau de bord interactif

Livrables

Livrable	Emplacement	Format
Dataset nettoyé	rapport/data/processed/weather_processed.csv	CSV (~11 colonnes)
Dataset brut	rapport/data/raw/weatherHistory.csv	CSV
Dump MongoDB	rapport/database/dump/	BSON (mongodump)
Code + infrastructure	github.com/Mourtalla-8/weather-DE-Project	Git + Docker
Tableau de bord	Application Streamlit (équipe Analyse)	Web interactif
Modèle prédictif	Random Forest — température (équipe Analyse)	Notebook / pickle

Conclusion générale

Ce projet a permis au Groupe de mettre en œuvre, de bout en bout, une chaîne complète d'ingénierie et de valorisation de la donnée, depuis l'extraction de données brutes hétérogènes jusqu'à la production d'un modèle prédictif opérationnel. L'équipe ETL a livré un pipeline complet, documenté et reproductible, reposant sur une infrastructure Docker combinant MinIO, Apache NiFi et MongoDB, capable de transformer un fichier brut de 96 453 relevés en une base de données propre, structurée et directement exploitable. Cette rigueur technique s'est traduite par la

correction méthodique de plusieurs classes de problèmes de qualité des données : valeurs manquantes sur le type de précipitations, doublons, et surtout erreurs de capteur sur la pression atmosphérique, un défaut qui, laissé non corrigé, aurait pu biaiser silencieusement l'ensemble des analyses en aval.

Sur cette base fiabilisée, l'équipe Analyse a pu conduire une exploration statistique rigoureuse, confirmant par des tests formels (corrélation de Pearson, analyse de variance) des relations physiquement attendues mais qu'il importait de vérifier plutôt que de supposer : un lien négatif fort entre température et humidité, une saisonnalité marquée et statistiquement significative, et des différences de comportement du vent selon le type de précipitations. Ces résultats ont ensuite été rendus accessibles à un public non technique grâce à un tableau de bord interactif développé avec Streamlit, illustrant la dernière étape trop souvent négligée de toute démarche data : la restitution.

L'aboutissement le plus significatif de cette seconde partie reste toutefois la construction d'un modèle de prédiction de la température, dont la précision obtenue (erreur moyenne absolue de 0,56 °C, coefficient de détermination de 0,993) démontre, mieux que toute autre analyse, la valeur du travail de préparation réalisé en amont : un modèle ne peut être performant que si les données qui l'alimentent sont fiables, complètes et correctement structurées. En ce sens, la qualité du modèle final constitue une forme de validation indirecte, mais concrète, de l'ensemble de la chaîne ETL présentée en première partie de ce rapport.

Au-delà de sa dimension pédagogique, un tel projet trouve un écho dans des enjeux concrets : une information météorologique plus accessible et mieux valorisée peut contribuer à améliorer la préparation aux événements climatiques, à soutenir la prise de décision dans le secteur agricole, ou plus simplement à aider chacun à mieux planifier ses activités quotidiennes. Le pipeline construit ici, bien que développé dans un cadre académique, repose sur des briques technologiques (Docker, MinIO, NiFi, MongoDB, scikit-learn, Streamlit) directement transposables à un contexte de production.

Les prochaines étapes de ce travail consisteront à intégrer les corrections demandées sur ce rapport, puis à préparer une présentation de synthèse destinée à restituer, de façon condensée et illustrée, l'ensemble de la démarche et des résultats présentés ici. À plus long terme, les pistes identifiées à l'issue de la partie modélisation — comparaison avec d'autres algorithmes, prévision à horizon plus long, déploiement opérationnel du modèle — constituent des prolongements naturels pour qui souhaiterait poursuivre ce projet au-delà de son cadre initial.

Références

Jeu de données : Historical hourly weather data — Szeged, Hungary (2006-2016), Kaggle (muthuj7/weather-dataset).

Dépôt de code : github.com/Mourtalla-8/weather-DE-Project